

Random forests

STAT 4710

October 28, 2025

Where we are

- ✓ **Unit 1:** R for data mining
- ✓ **Unit 2:** Prediction fundamentals
- ✓ **Unit 3:** Regression-based methods
- Unit 4:** Tree-based methods
- Unit 5:** Deep learning

Lecture 1: Growing decision trees

Lecture 2: Tree pruning and bagging

Lecture 3: Random forests

Lecture 4: Boosting

Lecture 5: Unit review and quiz in class

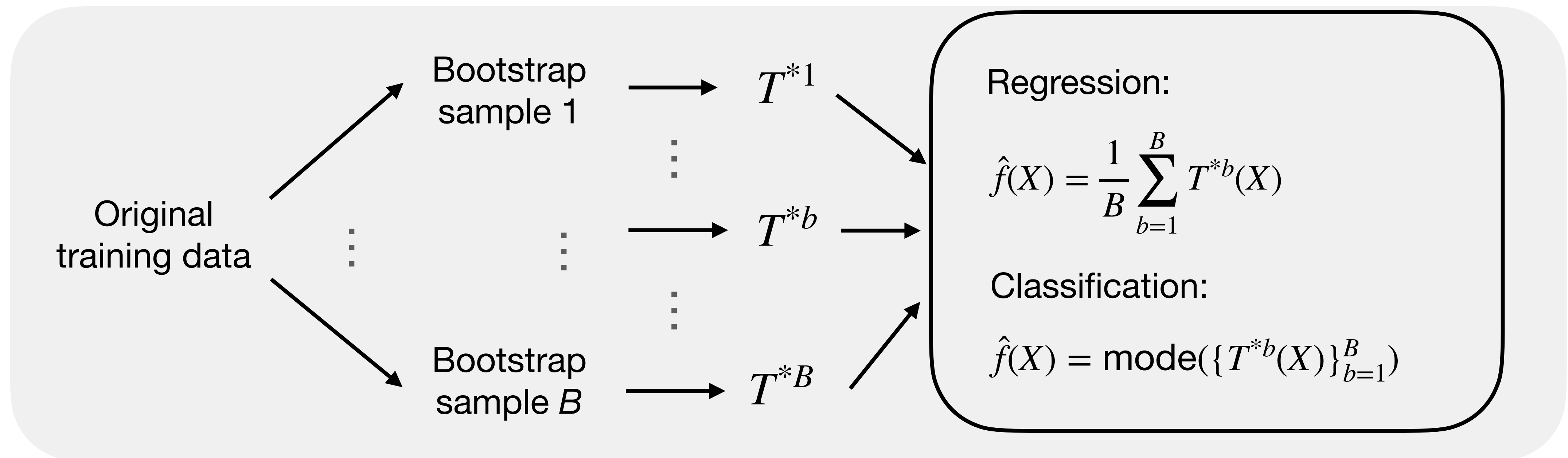
Plan for today

- **Limitations** of bagging in reducing variance
- **Solution: Random forests!**
What does it do, and how does it differ from bagging?
- **OOB error**: A nice alternative to cross-validation error for random forests
- **Interpretability**:
How to interpret the model when we no longer have a single tree to read?
Two approaches to measure feature importance

Key challenge of one single decision tree

A key challenge with one single decision tree is **stability**

Bagging: Average predictions from many bootstrap samples to reduce variance.



How much can bootstrapped trees help?

Variance reduction of bagging

The bagging prediction is defined by $\hat{f}(X) = \frac{1}{B} \sum_{b=1}^B T^{*b}(X)$.

Suppose $\text{Corr}[T^{*b_1}(X), T^{*b_2}(X)] = \rho \in [0,1]$. Then, we can derive that

$$\text{Var}[\hat{f}(X)] \approx \left(\frac{1}{B} + \frac{B-1}{B} \rho \right) \text{Var}[T(X)] \approx \rho \cdot \text{Var}[T(X)],$$

where $T(X)$ is a single decision tree.

- The variance is reduced by a factor of $\rho = \text{Corr}[T^{*b_1}(X), T^{*b_2}(X)]$, so the less **correlated** the bootstrapped trees prediction are, the better.
- The variance reduction depends on ρ , not B (once B is large enough)!
 $\leadsto$ Having more bootstrap samples doesn't really help

Random forests: More variance reduction

Random forests are the same as bagging, but with one key modification:

When constructing each tree, at each split point:

- Randomly sample a subset of $m \leq p$ features
- Split on the best feature *among this subset*

Intuition: Sampling subset of features at each split

- Decorrelates the trees (avoid similar trees)
- Reduces variance
- Boosts prediction performance

Note that setting $m = p$ recovers bagging.

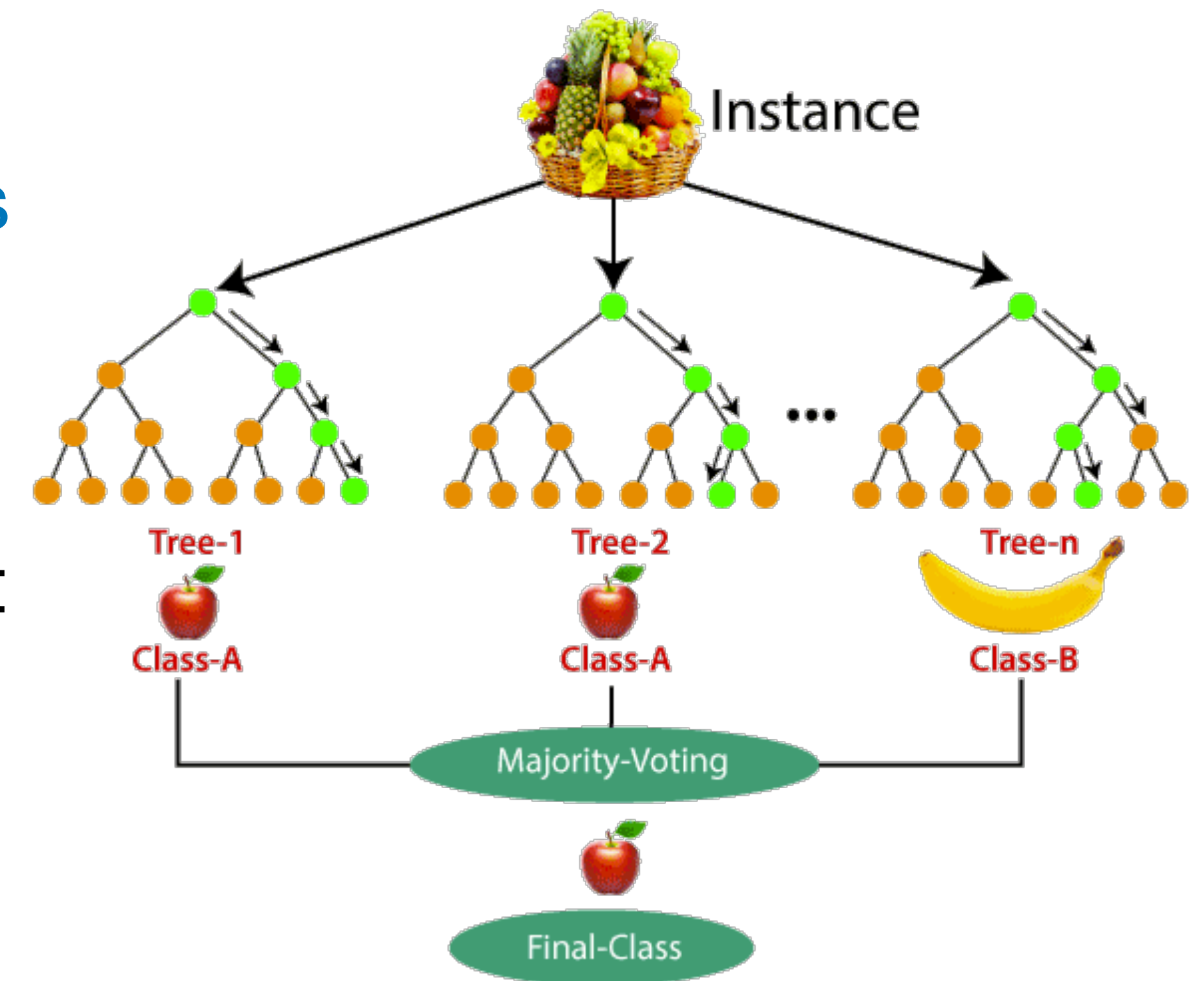


Image source: <https://www.section.io/engineering-education/introduction-to-random-forest-in-machine-learning/>

Random forests

Parameters:

- B : number of bootstrap samples
- m : number of variables sampled at each split
- criterion to stop splitting, like max number of nodes and/or min samples per node

Training:

- Extract B bootstrap samples from your training data
- For each bootstrap sample $b = 1, \dots, B$, grow a decision tree:
 - randomly sampling m features to split on at each step
 - until stopping criterion is met

Prediction:

- aggregate the B decision trees using the mean (for regression) or mode (for classification)

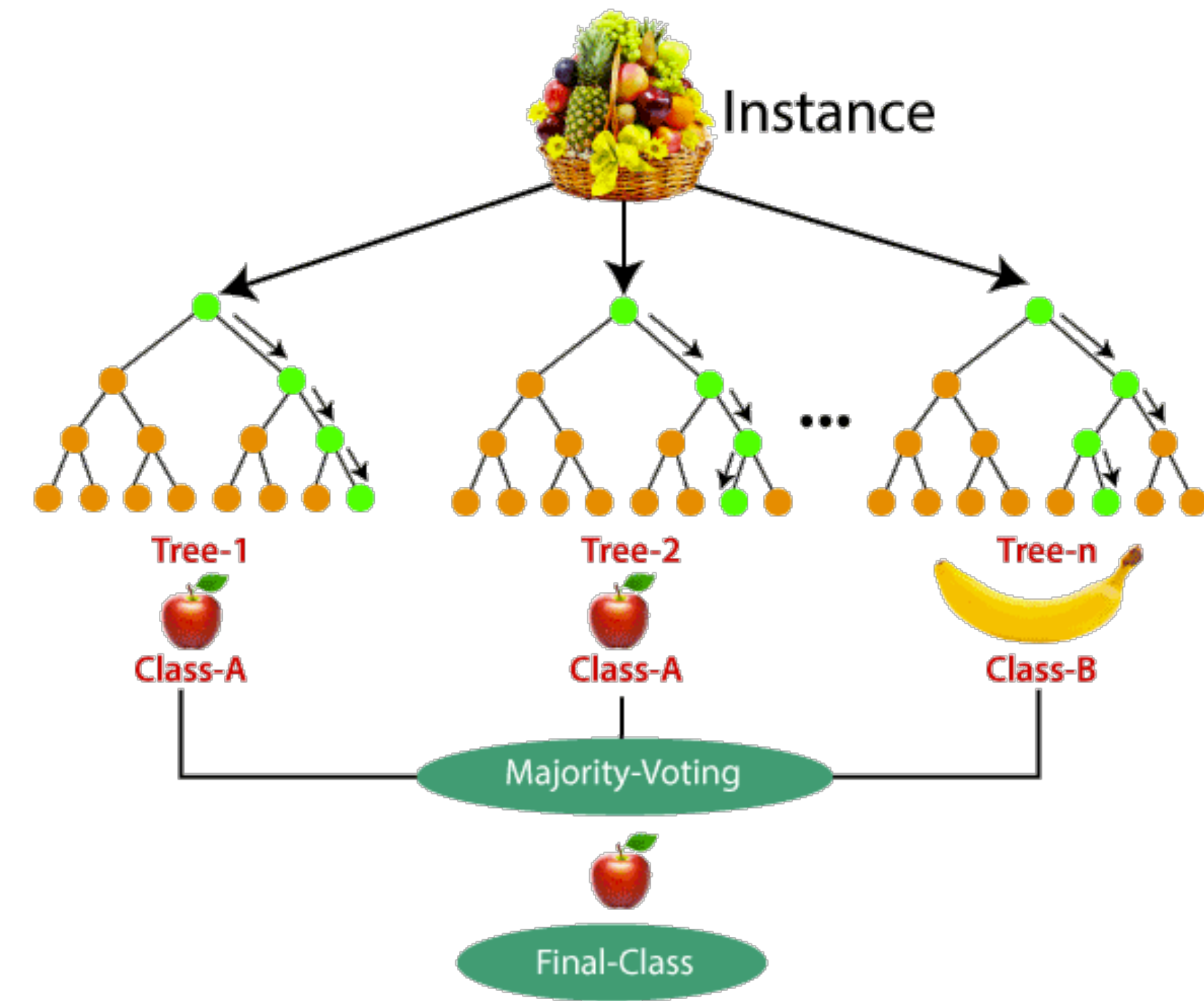


Image source: <https://www.section.io/engineering-education/introduction-to-random-forest-in-machine-learning/>

A bias-variance trade-off in choosing m

If m is larger, the random forest will have

- lower bias (it can better fit the underlying trend) but
- higher variance (more correlated trees)

If m is smaller, the random forest will have

- higher bias (not able to fit the truth, since only a small set of features are used) but
- lower variance (less correlated trees)

Default choices: $m = p/3$ for regression and $m = \sqrt{p}$ for classification.

For best predictive performance, m should be tuned.

Tuning random forests via out-of-bag error

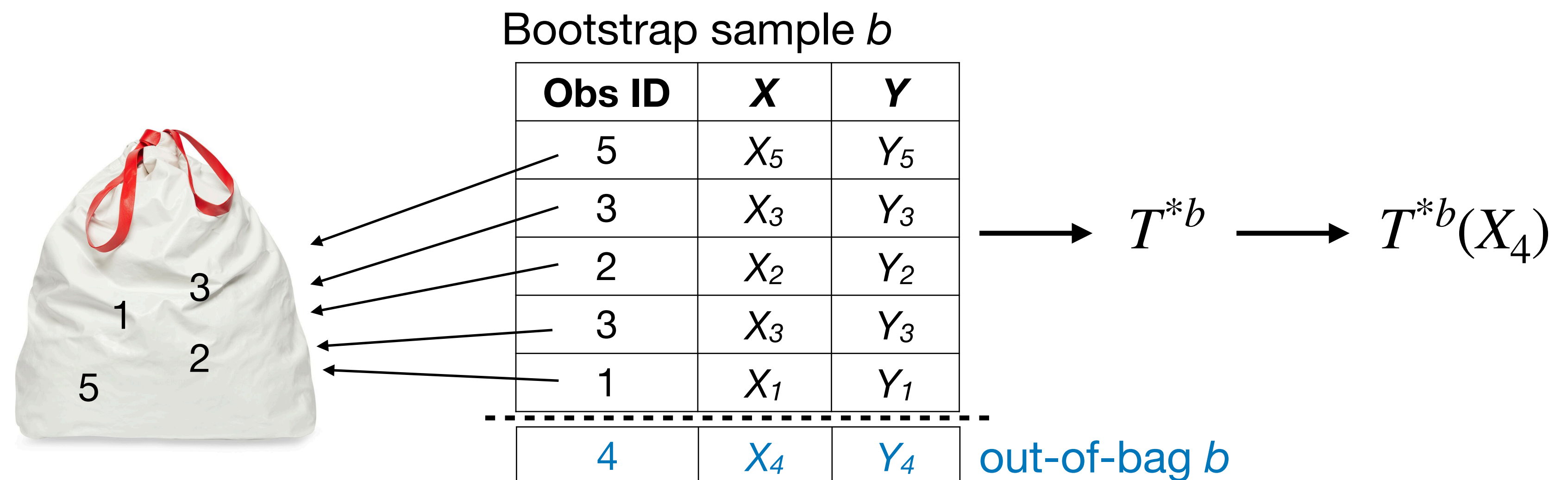
We usually tune prediction methods via cross-validation. For random forests, there is a clever and computationally faster alternative: **out-of-bag error**.

The idea behind CV is to use parts of training data as validation sets.

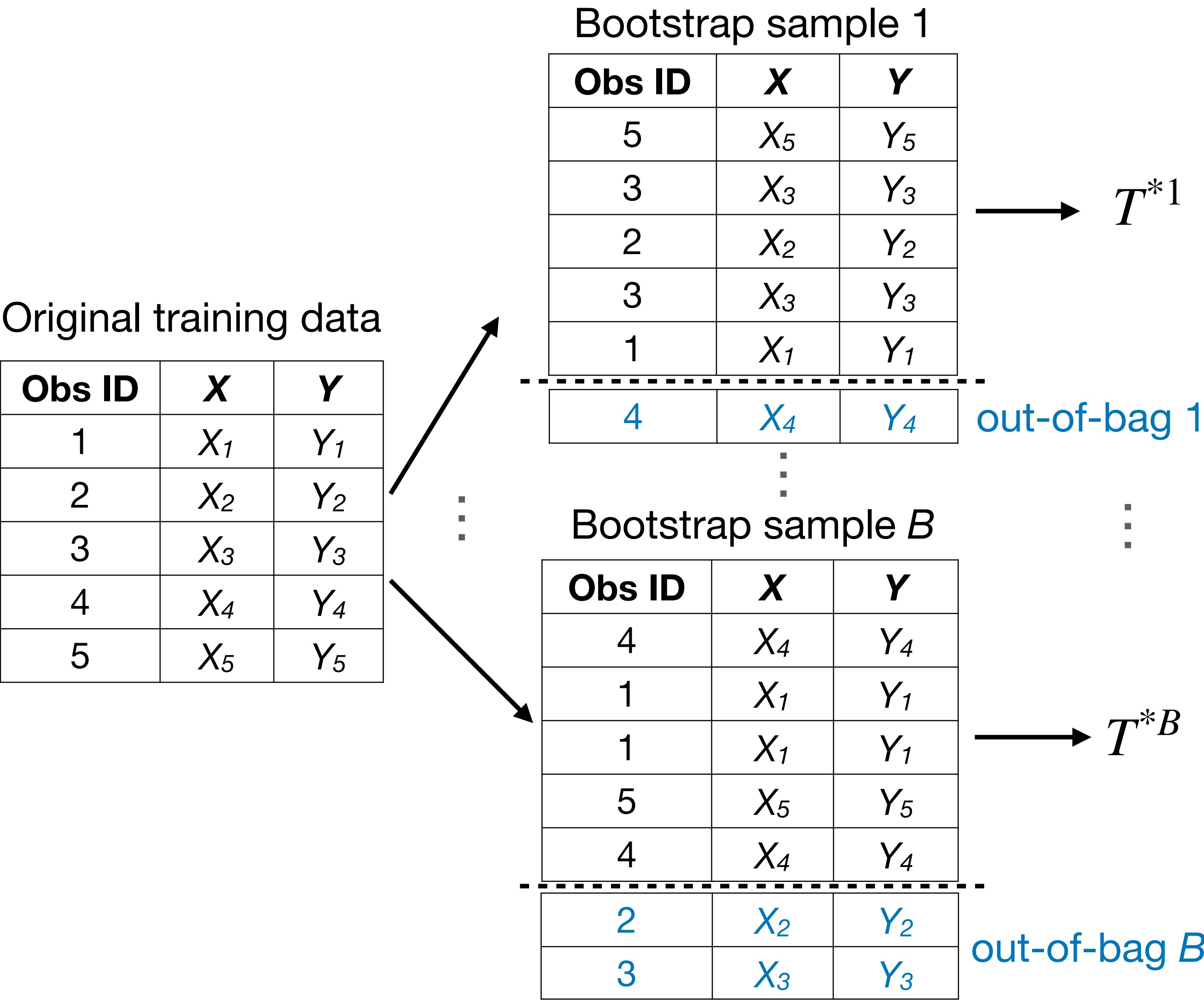
Bootstrapping already does this! [about 1/3 original data are not in bootstrap data]

For each bootstrap sample, define the “bag” to be the set of unique training observations in the sample.

Then, predictions based on that tree can be made on the out-of-bag (OOB) samples.



Out of bag error



OOB predictions

Obs ID	X	Y	$T^{*1}(X)$	...	$T^{*B}(X)$	$\hat{Y}^{OOB}$
1	X_1	Y_1	—	...	—	$\hat{Y}_1^{OOB}$
2	X_2	Y_2	—	...	$T^{*B}(X_2)$	$\hat{Y}_2^{OOB}$
3	X_3	Y_3	—	...	$T^{*B}(X_3)$	$\hat{Y}_3^{OOB}$
4	X_4	Y_4	$T^{*1}(X_4)$	...	—	$\hat{Y}_4^{OOB}$
5	X_5	Y_5	—	...	—	$\hat{Y}_5^{OOB}$

Regression:

$$\hat{Y}_i^{OOB} = \text{mean}\{T^{*b}(X_i)\}_{i \in \text{OOB}_b}$$

$$\text{OOB err} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i^{OOB})^2$$

Classification:

$$\hat{Y}_i^{OOB} = \text{mode}\{T^{*b}(X_i)\}_{i \in \text{OOB}_b}$$

$$\text{OOB err} = \frac{1}{n} \sum_{i=1}^n I(Y_i \neq \hat{Y}_i^{OOB})$$

Parameters to tune (or not)

Random forests generally work pretty well even if not tuned (i.e. if default parameter choices are used).

However, parameters can be tuned using OOB error to improve performance:

- *m*: most important tuning parameter
- *criteria to stop splitting*: can be tuned but growing trees about as deep as possible generally works pretty well
- *B*: least necessary to tune; just choose a large value like 100-1000.

Interpretability and variable importance measures

Compared to trees, main drawback of random forests is reduced interpretability.

However, **variable importance measures** can help improve the interpretability.

Variable importance: Which variables matter overall?

Two popular ideas of quantifying variable importance from trees:

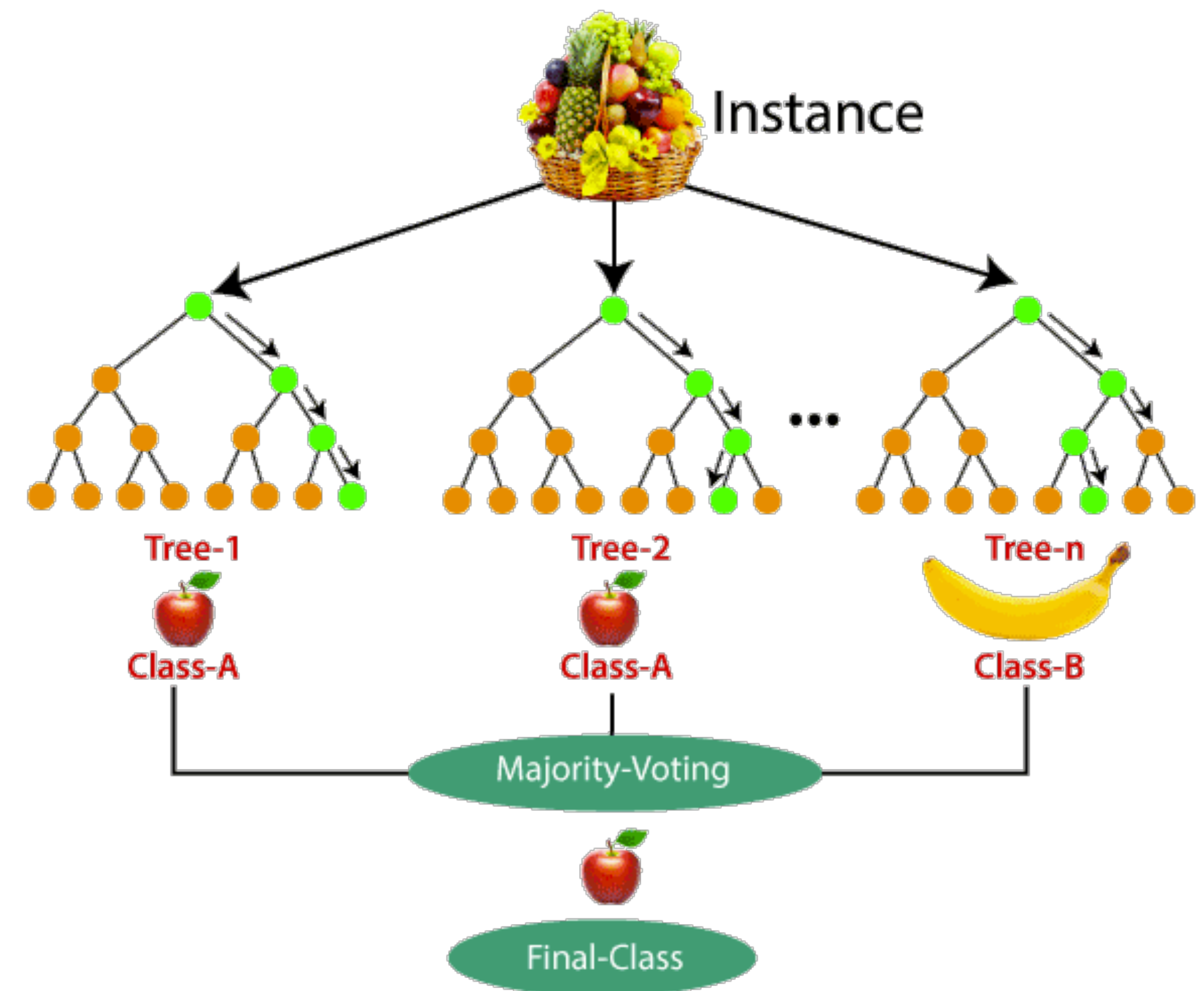
- **Purity-based:** Can I get a “pure”, clean cut split based on this variable?
- **Prediction based:** How much does “messing up this variable” changes my prediction?

Purity-based variable importance

Consider the construction of one tree. For each split, note the feature that was split on and **resulting reduction in RSS or Gini index** (i.e. improvement in purity).

Define the importance of each feature in this single tree by **summing up the improvement in purity for all splits based on this feature**.

For random forests, we can average this quantity over all of the trees to get a purity-based variable importance metric.



Purity-based variable importance

Intuition: splitting on an important variable should reduce RSS / Gini index



In each tree, look for the nodes that split on the variable



Compute the reduction in RSS/Gini in these nodes



Sum the reductions to measure importance



In forests, average over all the trees

OOB prediction based variable importance

Recall the OOB error introduced a few slides ago.

For each feature j and each tree, consider making predictions on the OOB data **after first scrambling feature j** . We can therefore get a **scrambled OOB error**.

For each feature j , we can define an OOB-prediction-based variable importance by **the difference in OOB error when this feature is scrambled and when it is not**.

$\mathbf{X} =$

	x_0	x_1	...	x_j	...	x_{p-1}
$\mathbf{X}_1$	12	0		a		1.5
$\mathbf{X}_2$	-3	1		b		-0.7
$\mathbf{X}_3$	5	0		c		0.2
$\mathbf{X}_4$	16	0		d		-3.5
$\mathbf{X}_5$	-7	1		e		0.9

$\xrightarrow{\text{scramble}}$

$\mathbf{X} =$

	x_0	x_1	...	x_j	...	x_{p-1}
$\mathbf{X}_1$	12	0		b		1.5
$\mathbf{X}_2$	-3	1		e		-0.7
$\mathbf{X}_3$	5	0		d		0.2
$\mathbf{X}_4$	16	0		c		-3.5
$\mathbf{X}_5$	-7	1		a		0.9

Regular OOB predictions

$T^{*1}(\mathbf{X})$	...	$T^{*B}(\mathbf{X})$	$\hat{Y}^{OOB}$
—	...	—	$\hat{Y}_1^{OOB}$
—	...	$T^{*B}(x_2)$	$\hat{Y}_2^{OOB}$
—	...	$T^{*B}(x_3)$	$\hat{Y}_3^{OOB}$
$T^{*1}(x_4)$	...	—	$\hat{Y}_4^{OOB}$
—	...	—	$\hat{Y}_5^{OOB}$

Regular
OOB error

Scrambled OOB predictions

$T^{*1}(\mathbf{X})$	...	$T^{*B}(\mathbf{X})$	$\hat{Y}^{OOB}$
—	...	—	$\hat{Y}_1^{OOB}$
—	...	$T^{*B}(x_2)$	$\hat{Y}_2^{OOB}$
—	...	$T^{*B}(x_3)$	$\hat{Y}_3^{OOB}$
$T^{*1}(x_4)$	...	—	$\hat{Y}_4^{OOB}$
—	...	—	$\hat{Y}_5^{OOB}$

Scrambled
OOB error

Variable Importance =

scrambled OOB err - regular OOB err

Prediction-based variable importance

Intuition: messing up an important variable should ruin the prediction



Scramble a variable in the data



Compute the OOB prediction error of the scrambled data



Measure the difference of OOB error compared with original



In forests, average over all the trees

Summary

- Random forests are a fancier version of bagging based on **random sub-sampling of m features** at each split point.
- They improve on bagging by **de-correlating** the bootstrapped decision trees and therefore **reducing the variance** of the method.
- OOB error is a nice alternative to cross-validation error for random forests, and can be used to tune parameters such as m .
- Random forests usually give much better prediction performance than individual decision trees, but at the cost of interpretability.
- Nevertheless, there are a couple ways to measure variable importance in random forests, giving us some interpretability.

Random forests are a state-of-the-art tool for predictive modeling.